Detailed Design Doc

Project Name: Social Impact Platform

Team Name: SIP6321

Client: Tracy Dansby

Repository Link: https://github.com/stevenyonduan/DCB6321-Social-Impact-Platform

Team Members

Member 1: Emilie Masson

Member 2: Brandon Shoemaker

Member 3: Amartya Kallingal

Member 4: Steven Yon

Table of Contents

Background 5

Document Summary 5

Static System Architecture 5

Data Storage Design 5

Static System Architecture 6

Introduction 6

Rationale 7

Security and Privacy 7

Static Architecture 8

Data Design 10

Introduction 10

Database Use 10

File Use 19

Data Exchange 19

Table of Figures

Figure 1 7
Figure 2 8
Figure 3 9
Figure 4 11
Figure 5 12
Figure 6 13
Figure 7 13
Figure 8 13
Figure 9 14
Figure 10 14
Figure 11 15
Figure 12 15
Figure 13 16
Figure 14 16
Figure 15 17
Figure 16 17
Figure 17 18

Terminology

API: Application Programming Interface; the functions available for programmatically interfacing with an application

Entity Table: a database table that stores the primary data of the application, such as users, posts, or events

ERD: Entity-Relationship Diagram; a diagram showing the tables, attributes, keys, and relationships of a database

JSON: JavaScript Object Notation; a lightweight data-interchange format

MVP: Minimum Viable Product; an initial version of the application with the core features needed for testing

PostgreSQL: an open-source relational database management system

React: a JavaScript library primarily used for building user interfaces for web applications

REST: Representational State Transfer; an architectural style for designing web APIs

SIP: Social Impact Platform; the volunteer discovery and social networking application described in this document

Spring Boot: a Java framework for building RESTful APIs and organizing application logic

Supabase: a backend platform that provides services such as a PostgreSQL database, authentication, and file storage

TypeScript: a programming language that adds static typing to JavaScriptUUID: Universally Unique Identifier; a unique identifier used for record IDs in the database

Vercel: a platform used to deploy and host the frontend application

Introduction

Background

The Social Impact Platform is a volunteering platform that connects young adults in Atlanta with nonprofit organizations offering volunteer opportunities. Volunteer opportunities are fragmented across organizations and platforms, making it difficult for volunteers and non-profits to connect. SOW wants a social platform to give volunteers an easy way to find non-profit organizations they connect with, along with the information needed to participate in different volunteering opportunities. In creating a social platform, SIP can create more engagement and interest around volunteering while connecting volunteers and causes through a dedicated social feed.

Document Summary

Static System Architecture

The System Architecture section describes the static structure of our system. The static system architecture diagram shows how the application is organized into a React + TypeScript frontend, a Spring Boot REST API, a set of backend services (Account + Profile, Opportunity + RSVP, Social Feed, Recommendation, and Volunteer Hours and Verification), and a data layer made up of PostgreSQL, Authentication, and File Storage.

Data Storage Design

The Data Storage Design section describes the database structure of our system, which consists of a PostgreSQL database and image storage on Supabase. We additionally provide an entity-relationship diagram for the database, a description of the data exchange between the frontend and the backend, and a section detailing how we secure the data of our application.

Static System Architecture

Introduction

The Social Impact Platform (SIP) aims to combine discovering volunteer opportunities with various aspects of social media platforms to create a centralized location for users to find their volunteer community. The user groups that we are focusing on are college students and young adults, as well as nonprofit organizations who will use the platform to publish their available volunteer opportunities. The main goals and objectives of this project are listed below:

- Centralized location for volunteer opportunities so users can browse, search, filter, and rsvp to opportunities they are interested in
- Support volunteer and nonprofit profiles and accounts. These profiles will in include volunteer interests that will help with personalized content
- Have a social experience around volunteering with features such as posts, feeds, following, likes, comments, and sharing
- Provide personalized recommendations for volunteer opportunities based on interests
- Track volunteer participation and support verification of submitted volunteer hours
- Provide admin capabilities
- Create an MVP that is testable, low cost, and easy to build on

Rationale

Technology	Architectural Role	Rationale
React + TypeScript	Presentation layer	Matches what the team has experience in and supports component based UI. Common frontend language ensuring the MVP is easy to build on
Java + Spring Boot	Rest API and business logic	The team has Java experience and Spring Boot gives us a good way to organize API endpoints and the logic behind the features
PostgresSQL	Relational Persistence	Most of the data is related so it makes the most sense to have a relational database
Supabase	Backend platform services	It gives us several backend services in one place and has a free tier that should be sufficient for the MVP
Vercel	Frontend deployment	It's simple enough to connect to a frontend project and makes it easy to publish updated versions

Figure 1

Security and Privacy

Authentication: Users will need to login to access the platform
Roles and Permissions: volunteers, admins, and non profits need to have different permissions. The backend and frontend need to check a user roles before allowing actions.
Private User Data: Users should only be able to access information that belongs to their account such as posts they save
Passwords: Passwords should be saved as just plain text in the database
Moderation: Admins need to be able to have admin access tom monitor users posts and comments

Static Architecture

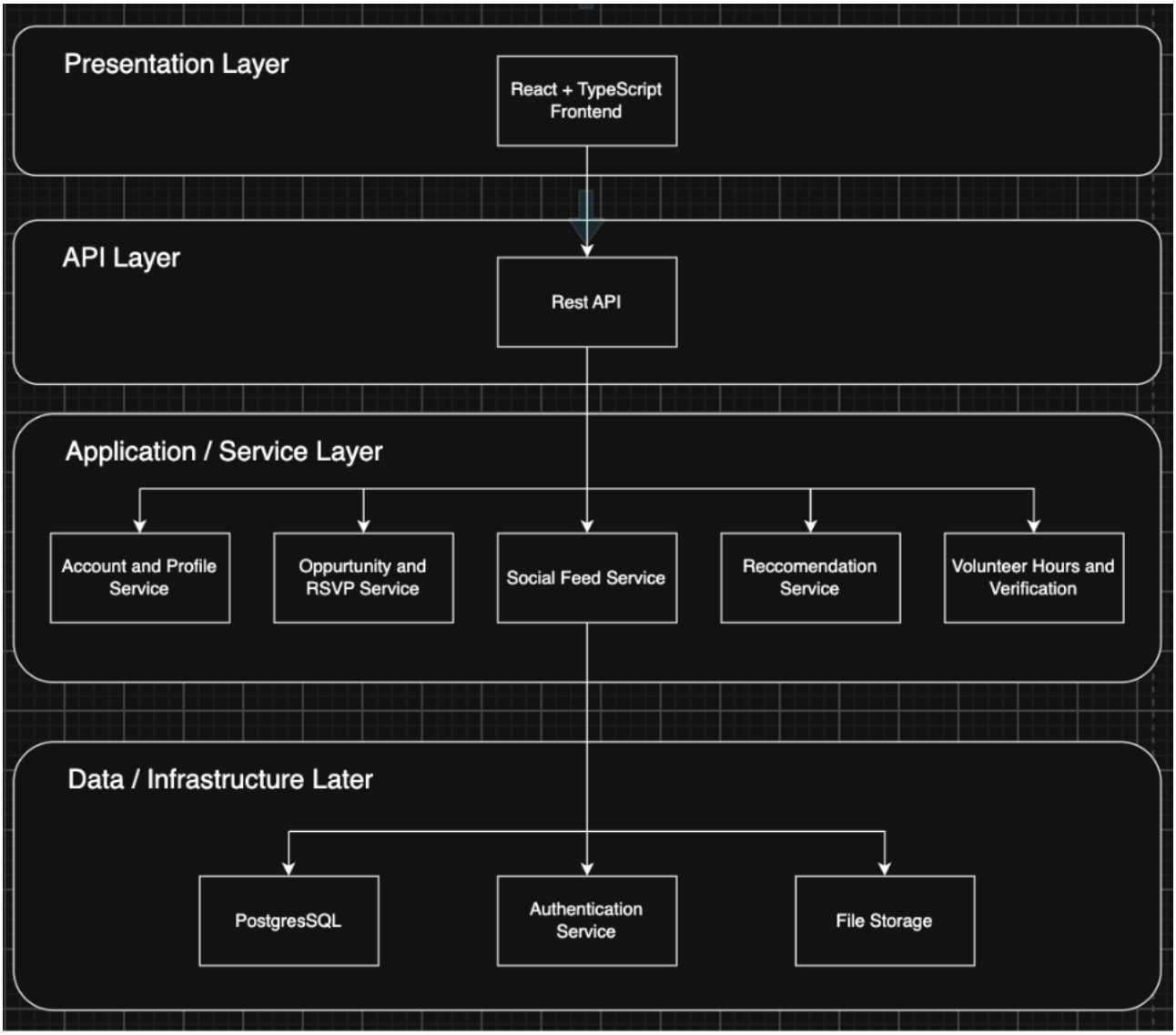


Figure 2

The SIP platform follows a layered architecture system consisting of a Presentation Later, API Layer, Application/Service Later and a Data/Infrastructure Later. The presentation later has the React frontend which will be the website interface. This depends on the Spring Boot REST API backend for its functionality. The API layer is the entry point to the backend and organizes some of the platforms major features. This layer depends on the Data Layer which stores different files and data for the platform.

Component	**Responsibility**
React + TypeScript Frontend	Website interface for profile, feed, and events
Spring Boot REST API	Backend API used by the frontend to request logic
Account + Profile Service	Handles volunteer and non profit profile information and interests
Oppurtunity + RSVP Service	Handles volunteer opportunity and RSVP functionality
Social Feed Service	Handles posts and social interactions
Recommendation Service	Uses profile interests and preferences to personalize content
Volunteer Hours and Verification Service	Handles submitted volunteer hours
PostgresSQL	Stores application data
Authentication	Supports login and other checks
File Storage	Stoes uploaded posts, and volunteer evidence

Figure 3

Data Design

Introduction

The SIP application will use PostgreSQL with Supabase, a relational database management system, to store and manage application data. The database will support user accounts, organizations, interests, social posts, events, volunteer participation, logged volunteer hours, and messaging. PostgreSQL was selected because the application contains multiple different entities that require relationships, while keeping organization and consistency.

Database Use

The database is organized into entity and junction tables. Entity tables store the application's primary data, including users, organizations, interests, posts, post comments, events, logged hours, and messages. Each entity table uses a primary key (PK) to uniquely identify its records. Foreign keys (FKs) establish relationships between tables, such as associating posts with their authors, events with their organizations, and volunteer hours with users

Junction tables implement many-to-many relationships between entities. For example, user_interests connects users to their selected interests, organization_members tracks user membership in organizations, and event_registrations connects users to events they register for. These junction tables use composite primary keys to prevent duplicate relationships and foreign keys to maintain referential integrity.

The database follows relational normalization principles by separating distinct entities into individual tables and using junction tables for many-to-many relationships. This reduces unnecessary data duplication and supports consistent updates across related records.

Figure 1 below presents the Entity-Relationship Diagram (ERD) for the SIP database, including its tables, attributes, primary and foreign keys, relationships, and cardinality constraints. Blue tables represent primary entities, while green tables represent junction tables. Underneath Figure 1 are the 13 tables describing the constraints of each entity’s attributes.

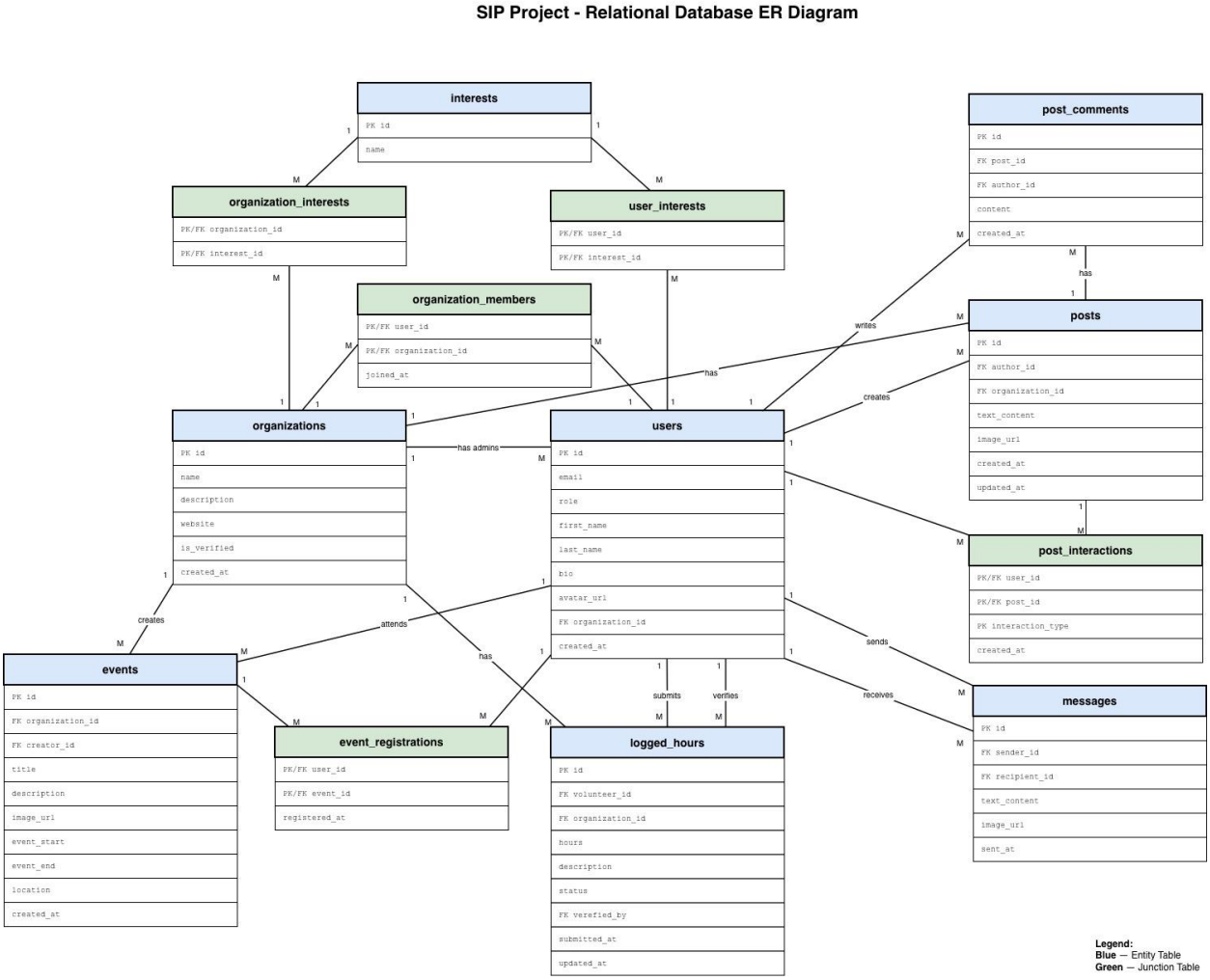


Figure 4

Figure 4. Relational Database ER Diagram

Figure 5: users (Stores profile data for all account types)

	Data Type	**Constraints**
id	UUID	PK Matches Supabase auth.users.id
email	VARCHAR(255)	UNIQUE NOT NULL
role	VARCHAR(20)	NOT NULL Enum: 'STANDARD', 'ADMIN', 'ORGANIZATION'
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
bio	TEXT	Nullable
avatar_url	VARCHAR(255)	Nullable Maps to user profile image
organization_id	UUID	FK references organizations(id) Nullable Enforces that Admins and Organization accounts are tied to exactly one organization
created_at	TIMESTAMPTZ	Default NOW()

Figure 5

Figure 6: organizations (Represents the volunteer entity itself)

	Data Type	**Constraints**
id	UUID	PK Default uuid_generate_v4()
name	VARCHAR(255)	NOT NULL
description	TEXT	Nullable
website	VARCHAR(255)	Nullable
is_verified	BOOLEAN	Default FALSE For verifying organization authenticity
created_at	TIMESTAMPTZ	Default NOW()

Figure 6

Figure 7: interests (Lookup table for standardizing tags)

	Data Type	**Constraints**
id	UUID	PK
name	VARCHAR(255)	UNIQUE NOT NULL

Figure 7

Figure 8: user_interests (Junction table for user interests used for personalized recommendations and filtering)

	Data Type	**Constraints**
user_id	UUID	PK, FK references users(id)
interest_id	UUID	PK, FK references interests(id)

Figure 8

Figure 9: organization_interests (Junction table for organization interests/categories used for personalized recommendations and filtering)

	Data Type	**Constraints**
organization_id	UUID	PK, FK references organizations(id)
interest_id	UUID	PK, FK references interests(id)

Figure 9

Figure 10: organization_members (Tracks volunteers joining specific organizations/groups)

	Data Type	**Constraints**
user_id	UUID	PK, FK references users(id)
organization_id	UUID	PK, FK references organizations(id)
joined_at	TIMESTAMPTZ	Default NOW()

Figure 10

Figure 11: posts (Supports text and image content for the community feed)

	Data Type	**Constraints**
id	UUID	PK
author_id	UUID	FK references users(id) NOT NULL
organization_id	UUID	FK references organizations(id) Nullable Populated if the post is explicitly tied to a group's page
text_content	TEXT	Nullable; application logic must enforce that either text or image is present
image_url	VARCHAR(255)	Nullable; application logic must enforce that either text or image is present
created_at	TIMESTAMPTZ	Default NOW()
updated_at	TIMESTAMPTZ	Tracks post edits

Figure 11

Figure 12: post_interactions (Tracks user likes and saves)

	Data Type	**Constraints**
user_id	UUID	PK, FK references users(id)
post_id	UUID	PK, FK references posts(id)
interaction_type	VARCHAR(20)	PK Enum: 'LIKE', 'SAVE'
created_at	TIMESTAMPTZ	Default NOW()

Figure 12

Figure 13: post_comments (Text-only responses to posts)

	Data Type	**Constraints**
id	UUID	PK
post_id	UUID	FK references posts(id) NOT NULL
author_id	UUID	FK references users(id) NOT NULL
content	TEXT	NOT NULL
created_at	TIMESTAMPTZ	Default NOW()

Figure 13

Figure 14: events (Volunteering opportunities created by organizations)

	Data Type	**Constraints**
id	UUID	PK
organization_id	UUID	FK references organizations(id) NOT NULL
creator_id	UUID	FK references users(id) Tracks which admin created it
title	VARCHAR(255)	NOT NULL
description	TEXT	NOT NULL
image_url	VARCHAR(255)	Nullable
event_start	TIMESTAMPTZ	NOT NULL
event_end	TIMESTAMPTZ	NOT NULL
location	VARCHAR(255)	NOT NULL
created_at	TIMESTAMPTZ	Default NOW()

Figure 14

Figure 15: event_registrations (Tracks RSVPs)

	Data Type	**Constraints**
user_id	UUID	PK, FK references users(id)
event_id	UUID	PK, FK references events(id)
registered_at	TIMESTAMPTZ	Default NOW()

Figure 15

Figure 16: logged_hours (Manual hour submissions requiring admin verification)

	Data Type	**Constraints**
id	UUID	PK
volunteer_id	UUID	FK references users(id) NOT NULL
organization_id	UUID	FK references organizations(id) NOT NULL Defines who must approve it
hours	NUMERIC(5,2)	NOT NULL
description	TEXT	Nullable
status	VARCHAR(20)	NOT NULL; Default 'PENDING'; Enum: 'PENDING', 'APPROVED', 'DENIED'
verified_by	UUID	FK references users(id); NULLABLE
submitted_at	TIMESTAMPTZ	Default NOW()
updated_at	TIMESTAMPTZ	Timestamp of approval/denial

Figure 16

Figure 17: messages (Basic direct messaging functionality)

	Data Type	**Constraints**
id	UUID	PK
sender_id	UUID	FK references users(id) NOT NULL
recipient_id	UUID	FK references users(id) NOT NULL
text_content	TEXT	Nullable Application logic must enforce that either text or image is present
image_url	VARCHAR(255)	Nullable Application logic must enforce that either text or image is present
sent_at	TIMESTAMPTZ	Default NOW()

Figure 17

File Use

The SIP application will support image files uploaded by users and organizations. These files will include user profile pictures, images attached to social posts, and images used for events. Supported image formats will include JPEG and PNG.

Uploaded images will be stored using Supabase Storage, while the PostgreSQL database will store references to these files rather than the image files themselves. For example, the avatar_url field in the users table, the image_url field in the posts table, and the image_url field in the events table will reference their corresponding images.

This approach keeps large image files separate from the relational database and allows the application to retrieve and display images when needed. File uploads will be subject to file type and size validation to help prevent unsupported or excessively large uploads.

Data Exchange

The SIP application will exchange data between the frontend, backend, and Supabase services. The frontend will send requests to the backend using a RESTful API over HTTPS. Data will be exchanged primarily in JSON format, allowing the frontend and backend to communicate using a consistent structure.

The backend will process requests, apply application logic, and interact with the Supabase PostgreSQL database to retrieve, create, update, or delete records. For image uploads, the application plans to use Supabase Storage to store image files separately from database records.

To protect user information, all communication will use HTTPS to encrypt data during transmission. Supabase Authentication is planned to manage user login and password security. Access controls will restrict sensitive operations based on user roles and permissions, such as allowing authorized organization accounts to manage events and verify volunteer hours. Personal information, including user profiles and messages, will be protected through appropriate authentication and authorization checks.